



Security Audit Report



Witness Chain - OperatorRegistry

Version: Final

Date: 10 July 2024

Table of Contents

Table of Contents	2
License	3
Disclaimer	4
Introduction	5
Purpose of this report	5
Codebases Submitted for the Audit	6
How to Read This Report	7
Overview	8
Summary of Findings	9
Detailed Findings	10
1. Reuse of Signatures	10
2. Incorrect usage of super keyword	12

License

THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE](#).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED "AS IS", WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

Introduction

Purpose of this report

OxCommit has been engaged by **Witness Chain - OperatorRegistry** to perform a security audit of several Smart Account components.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behaviour.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebases Submitted for the Audit

The audit has been performed on the following GitHub repositories:

Version	Commit hash	Github link
1	3546fc9150b0354be763f7bd9cefb0a6573a29ca	https://github.com/witnesschain-com/diligence-watchtower-contracts/blob/3546fc9150b0354be763f7bd9cefb0a6573a29ca/src/core/OperatorRegistry.sol

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Overview

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line by line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation

Summary of Findings

Sr. No.	Description	Severity	Status
1	Reuse of Signature	High ▾	Resolved ▾
2	Incorrect use of super keyword	Low ▾	Resolved ▾

Detailed Findings

1. Reuse of Signatures

Severity: **High** ▾

Description

In the **registerWatchtowerAsOperator** function, a signed message by a **watchtower**, along with an **expiration** timestamp, is accepted. This allows an allowed operator to transmit the message and gain access as an operator for a **watchtower**. However, the signature can be reused multiple times within the duration of the expiration, making it susceptible to repudiation attacks. This undermines the effectiveness of the **deRegister** function if the watchtower has provided a signature with a longer expiration to an operator. For instance, the operator can repeatedly use **deregister** and **registerWatchtowerAsOperator** as long as the signature for the respective watchtower has not expired, potentially misusing this behavior in a larger scheme.

To mitigate this vulnerability, signatures should ideally be single-use and associated with a nonce in addition to an expiration timestamp.

Remediation

Following is the remediated code -

Unset

```
// OperatorRegistry.sol - Remediated code

    // CHANGES DONE
    mapping ( address => mapping ( uint256 => bool ) ) private nonces;
    // CHANGES DONE

    function calculateWatchtowerRegistrationMessageHash(address operator, uint256
nonce, uint256 expiry)
    public
    pure
    returns (bytes32)
    {
        // CHANGES DONE
        // calculate the struct hash
        bytes32 structHash = keccak256(abi.encode(operator,nonce, expiry));
        // CHANGES DONE

        // get the ethSignedMessageHash
        bytes32 ethSignedMessageHash = structHash.toEthSignedMessageHash();
```

```

        return ethSignedMessageHash;
    }
    function registerWatchtowerAsOperator(address watchtower, uint256 expiry, uint256
nonce, bytes memory signedMessage)
        external
        whenNotPaused
    {
        // check if operator is whitelisted with Witness Chain
        require(whitelisted[msg.sender], "WitnessHub: Operator is not whitelisted with
Witness Chain AVS");
        // check if operator has previously registered for delegation with EigenLayer
        if (checkIsDelegatedOperator) {
            bool isDelegatedOperator =
IDelegationManager(delegationManagerAddress).isOperator(msg.sender);
            require(isDelegatedOperator, "WitnessHub: You need to be a delegated
operator with EigenLayer");
        }
        // check if the watchtower address is already registered
        require(watchtowerToOperator[watchtower] == address(0), "WitnessHub:
Watchtower address already registered");
        // check if 0 address is being registered as watchtower (disallow)
        require(watchtower != address(0), "WitnessHub: Watchtower address cannot be
the 0 address");
        // check if it's past expiry time already
        require(expiry >= block.timestamp, "WitnessHub: watchtower signature
expired");
        // CHANGES DONE
        // check if it's nonce is already used
        require(nonces[watchtower][nonce] == false, "WitnessHub: watchtower nonce
already used");
        nonces[watchtower][nonce] = true;
        // CHANGES DONE

        // validate the watchtower's signature to the registration
        bytes32 registrationStructHash =
calculateWatchtowerRegistrationMessageHash(msg.sender, nonce, expiry);
        //_validateWatchtowerRegistrationSignature(watchtower, registrationStructHash,
signedMessage);
        bool isValidSignature = SignatureChecker.isValidSignatureNow(watchtower,
registrationStructHash, signedMessage);
        require(
            isValidSignature,
            "OperatorRegistry.registerWatchtowerAsOperator: Invalid Signature"
        );
        _register(watchtower);
    }

```

Status

Resolved ▾

2. Incorrect usage of super keyword

Severity: **Low** ▾

Description

The **super** keyword in Solidity serves to explicitly call functions or access state variables from a parent contract within an inheritance hierarchy. Best practice dictates using the **super** keyword when inheriting a function from a child contract and adding additional functionality to it. However, in the case of the **pause** and **unpause** functions where the function names do not match those of inherited functions, there is no need for **super** usage. Instead, one can directly call the **_pause** and **_unpause** functions.

Remediation

Use simple calls to **_pause** and **_unpause** in respective functions.

Status

Resolved ▾